МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Отчет по выполнению практического задания №1

Тема: ЭМПИРИЧЕСКИЙ АНАЛИЗ СЛОЖНОСТИ АЛГОРИТМОВ

Дисциплина: Структуры и алгоритмы обработки данных

Выполнил студент Ноздрин Н.А.
группа ИКБО-17-25

Москва 2026

Цель работы: актуализация знаний и приобретение практических умений и навыков по определению вычислительной сложности алгоритмов (эмпирический под ход).

Задание 1.

Определить эффективный алгоритм из двух предложенных, используя оценку практической сложности каждого из алгоритмов и их ёмкостную сложность.

Пусть имеется задача: дан массив из n элементов символьного типа (С-строка), удалить из массива все значения (символы), равные заданному пользователем (ключевому).

Алгоритм 1.

Математическая модель решения:

При обнаружении ключевого элемента алгоритм сдвигает все последующие элементы влево на одну позицию, уменьшая размер массива.

```
delFirstMetod(x,n,key) {
i←1
while (i<=n) do
        if x[i]=key then
                //удаление
                 for j←i to n-1 do
                 x[j] ←x[j+1]
                od
                n←n-1
        else
                i←i+1
        endif
od
}
```

Код программы:

```
void delFirstMetod(int x[], int &n, int key) {
    int i = 0;
    while (i < n) {
        if (x[i] == key) {
            for (int j = i; j < n - 1; j++) {
                x[j] = x[j + 1];
            }
            n = n - 1;
        } else {
            i = i + 1;
        }
    }
}
```

Результаты тестирования (лучший случай):

n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.00080	100	0	100
200	0.00050	200	0	200
500	0.00070	500	0	500
1000	0.00130	1000	0	1000
2000	0.00600	2000	0	2000
5000	0.01360	5000	0	5000
10000	0.02830	10000	0	10000
100000	0.27600	100000	0	100000

Результаты тестирования (средний случай):

n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.00140	100	117	217
200	0.00200	200	553	753
500	0.01340	500	3814	4314
1000	0.04640	1000	16744	17744
2000	0.09100	2000	70458	72458
5000	0.91630	5000	500761	505761
10000	3.42780	10000	2030418	2040418
100000	193.07990	100000	195938157	196038157

Результаты тестирования (худший случай):

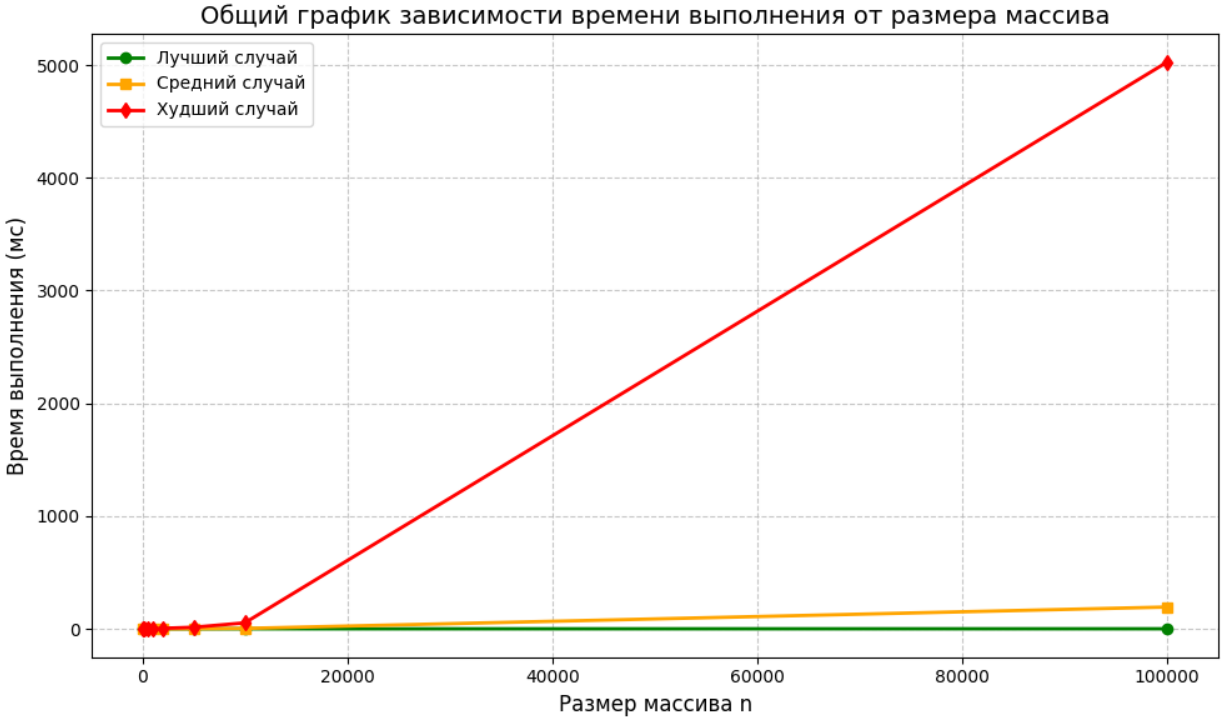
n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.01260	100	4950	5050
200	0.07730	200	19900	20100
500	0.31710	500	124750	125250
1000	0.80700	1000	499500	500500
2000	4.33000	2000	1999000	2001000
5000	13.91480	5000	12497500	12502500
10000	52.86800	10000	49995000	50005000
100000	5026.28580	100000	4999950000	5000050000

Ёмкостная сложность алгоритмов:

- **Лучший случай:** Временная сложность **O(n)**, доп. память **O(1)**. Возникает, если ключ в массиве не найден: алгоритм просто один раз проходит по всем элементам, ничего не перемещая.
- **Средний случай:** Временная сложность **O(n^2)**, доп. память **O(1)**. Если часть элементов совпадает с ключом, их удаление вызывает многократные сдвиги оставшейся части массива.
- **Худший случай:** Временная сложность **O(n^2)**, доп. память **O(1)**. Если все элементы массива подлежат удалению (равны ключу), выполняется максимально возможное количество сдвигов – n * (n - 1) / 2.

.

График зависимости времени решения задачи (в мс) для трёх случаев:



Вывод:

Лучший случай наблюдается тогда, когда в массиве отсутствуют элементы, равные ключу, поскольку алгоритм выполняет один проход без сдвигов элементов. Средний случай соответствует случайному расположению ключевых элементов, при котором часть элементов удаляется и возникают повторяющиеся сдвиги. Худший случай реализуется тогда, когда все

элементы массива равны ключу, так как после каждого удаления приходится сдвигать оставшуюся часть массива, что приводит к квадратичному росту

Алгоритм 2

Математическая модель решения:

Алгоритм выполняет один линейный проход по исходному массиву. Используются два индекса: i -для просмотра всех элементов исходного массива, j -для записи в начало массива только тех элементов, которые не равны ключу. Если очередной элемент x[i] не совпадает с key, он сохраняется в позиции x[j], после чего индекс j увеличивается на единицу. Если элемент равен ключу, он пропускается. После завершения прохода значение j определяет новое эффективное количество элементов массива, поэтому новая длина принимается равной n = j. Порядок неключевых элементов при этом сохраняется.

```
delOtherMetod(x,n,key) {
j←1
for i←1 to n do
   x[j]=x[i];
   if x[i]!=key then
      j++
   endif
od
if x[1]=key then
   n←0
else
   n←j
endif
}
```

Код программы:

```
void delOtherMetod(int x[], int &n, int key) {
    int j = 0;
    for (int i = 0; i < n; i++) {
        x[j] = x[i];
        if (x[i] != key) {
            j++;
        }
    }

    if (n > 0 && x[0] == key) {
        n = 0;
    } else {
        n = j;
    }
}
```

Результаты тестирования (лучший случай):

n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.00090	100	100	200
200	0.00160	200	200	400
500	0.00240	500	500	1000
1000	0.00590	1000	1000	2000
2000	0.00980	2000	2000	4000
5000	0.02380	5000	5000	10000
10000	0.04440	10000	10000	20000
100000	0.26720	100000	100000	200000

Результаты тестирования (средний случай):

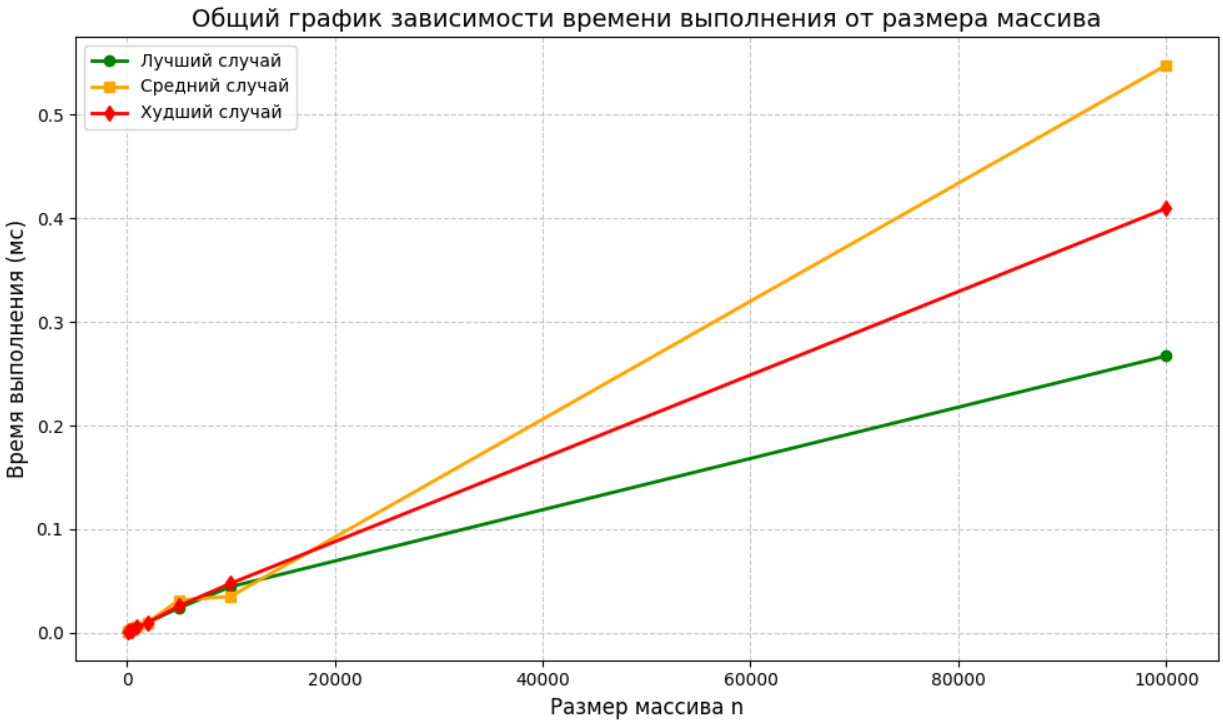
n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.00120	100	100	200
200	0.00160	200	200	400
500	0.00310	500	500	1000
1000	0.00630	1000	1000	2000
2000	0.00970	2000	2000	4000
5000	0.03130	5000	5000	10000
10000	0.03510	10000	10000	20000
100000	0.54780	100000	100000	200000

Результаты тестирования (худший случай):

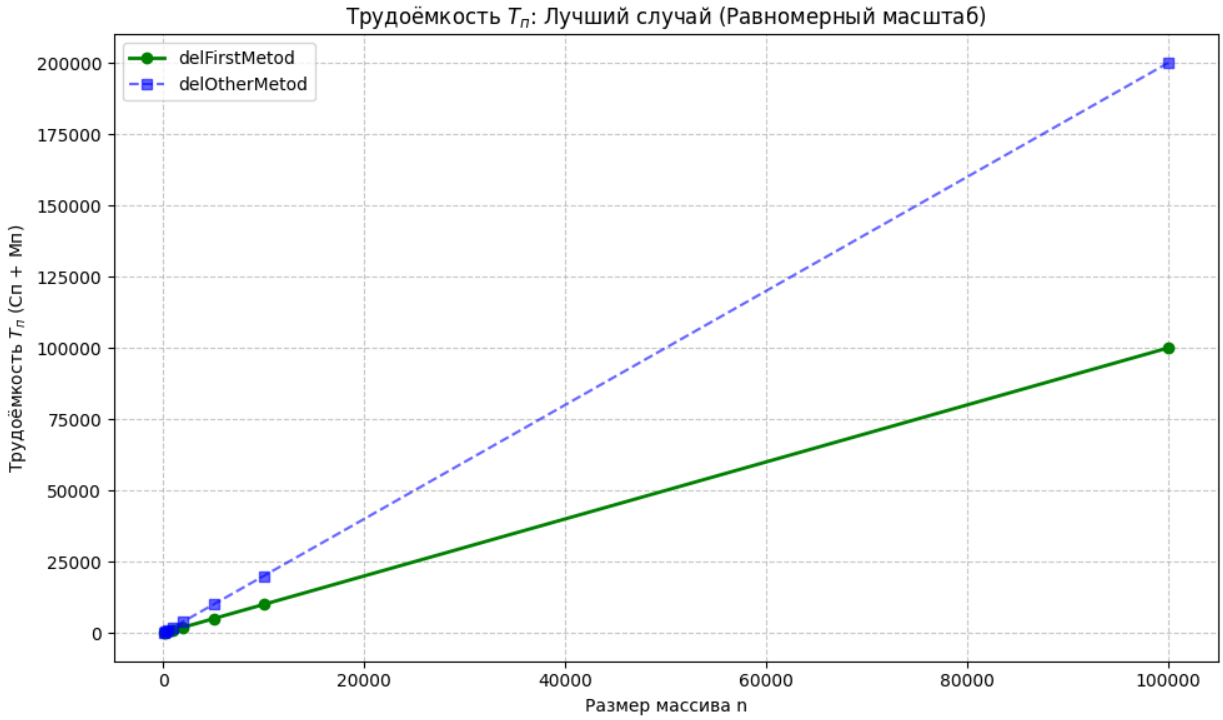
n	Время(мс)	Сп	Мп	Тп = Сп + Мп
100	0.00120	100	100	200
200	0.00190	200	200	400
500	0.00200	500	500	1000
1000	0.00520	1000	1000	2000
2000	0.00960	2000	2000	4000
5000	0.02570	5000	5000	10000
10000	0.04760	10000	10000	20000
100000	0.40970	100000	100000	200000

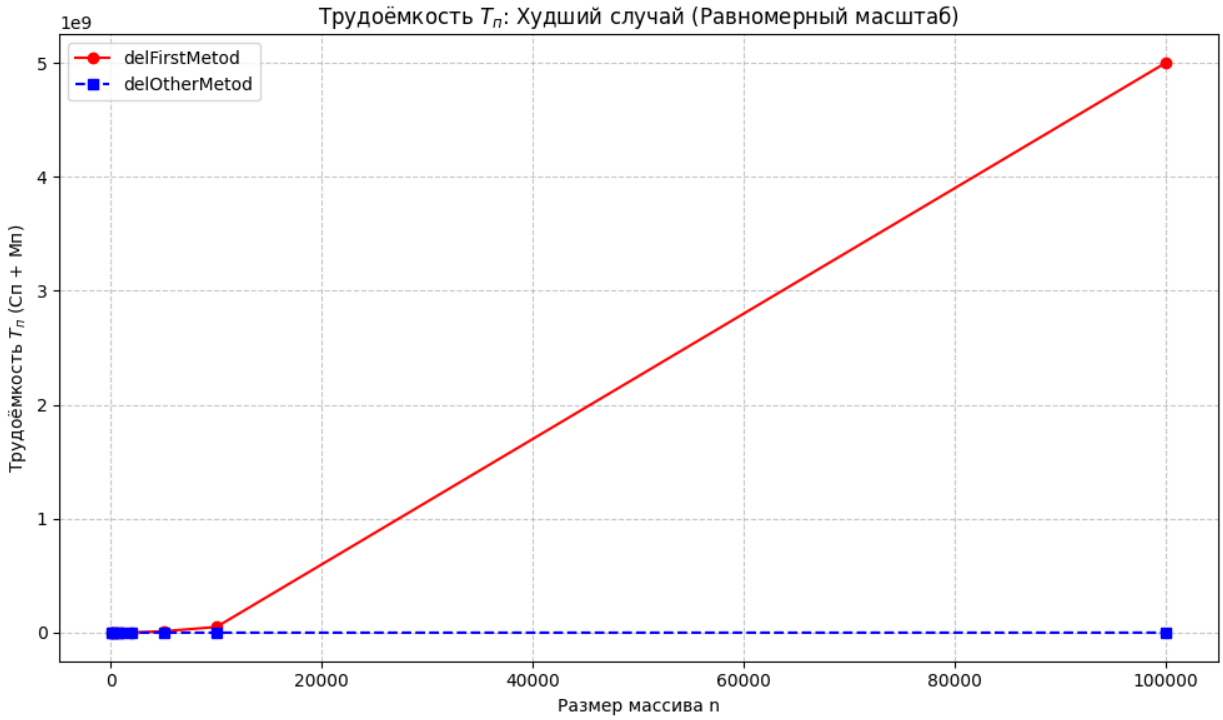
- **Лучший случай:**
  - Временная сложность: O(n)
  - Дополнительная память: O(1)
  - Пояснение: Выполняется один линейный проход по массиву.
- **Средний случай:**
  - Временная сложность: O(n)
  - Дополнительная память: O(1)
  - Пояснение: Положение ключей не меняет линейный характер работы алгоритма.
- **Худший случай:**
  - Временная сложность: O(n)
  - Дополнительная память: O(1)
  - Пояснение: Даже если удаляются все элементы, алгоритм завершается за один проход по массиву.

График зависимости времени решения задачи (в мс) для трёх случаев:



Сравнительные графики зависимости сразу для обоих алгоритмов:





Способ приведения массива к эффективной длине:

После удаления ключевых элементов фактическое количество полезных символов в массиве уменьшается, однако исходный объём выделенной памяти остаётся прежним. Для более экономного использования памяти можно создать новый динамический массив длиной, равной эффективному числу оставшихся элементов, скопировать в него полезные данные, освободить память старого массива и заменить старый указатель новым. Такой подход не влияет на результат удаления, но позволяет привести фактически выделенную память в соответствие с новой длиной массива.

Создание нового массива в C++:

```
char* newArr = new char[n];

memcpy(newArr, x, n);

delete[] x;

x = newArr;
```

Выводы по заданию 1

• **Лучший случай:** для **delFirstMethod** -O(n), доп. память O(1); для **delOtherMethod** - O(n), доп. память O(1). В лучшем случае оба алгоритма линейные, но второй метод также не требует дополнительной памяти сверх константной.

• **Средний случай:** для **delFirstMethod** -$O(n^2)$, доп. память O(1); для **delOtherMethod** -O(n), доп. память O(1). При случайном распределении удаляемых элементов второй метод заметно эффективнее первого.

• **Худший случай:** для **delFirstMethod** -$O(n^2)$, доп. память O(1); для **delOtherMethod** -O(n), доп. память O(1). При максимальном числе удалений второй метод значительно лучше, так как не выполняет многократные сдвиги элементов.

• **Поведение Тп(n):** У **delFirstMethod** наблюдается быстрый рост сложности в среднем и особенно в худшем случае; у **delOtherMethod** -линейный рост во всех случаях. Графики подтверждают теоретические оценки сложности.

• **Практическая пригодность: delFirstMethod** подходит только в ситуациях, когда удалений мало или они отсутствуют; **delOtherMethod** предпочтителен для общего случая. Для реального использования второй метод надежнее и предсказуемее.

Вывод по второму алгоритму:

Для данного алгоритма лучший, средний и худший случаи имеют одинаковый линейный характер, поскольку независимо от распределения ключевых элементов выполняется один проход по массиву. Различие между случаями может проявляться только незначительно во времени выполнения на практике, однако количество основных действий растёт пропорционально размеру массива. Следовательно, алгоритм 2 обладает линейной временной сложностью во всех трёх исследованных ситуациях.

Вывод по заданию 1:

По результатам эмпирического исследования установлено, что алгоритм **delFirstMethod** эффективен только в лучшем случае, когда удаление элементов не выполняется и трудоёмкость остаётся линейной. При случайном расположении ключевых элементов и особенно при удалении всех

элементов его трудоёмкость быстро возрастает из-за многократных сдвигов, что соответствует квадратичной зависимости. Алгоритм **delOtherMethod** во всех трёх случаях выполняет один линейный проход по массиву, поэтому является более эффективным по времени в общем случае. По расходу дополнительной памяти оба алгоритма эквивалентны, так как используют только постоянное количество дополнительной памяти O(1)

Задание 2 и Задание 3

Для выполнения задания выбран алгоритм Простого обмена («пузырьковая» сортировка, Bubble Sort)

Формулировка задачи:

Оценить сложность алгоритма сортировки простым обменом на массиве, заполненном случайными числами (средний случай).

Математическая модель решения:

Алгоритм сортировки простым обменом («пузырьковая» сортировка) - это алгоритм, который многократно проходит по массиву, сравнивая соседние элементы. Массив условно делится на две части:

- Отсортированная часть -находится в конце массива, первоначально пуста
- Неотсортированная часть -содержит оставшиеся элементы в начале массива

Алгоритм работает следующим образом:

1. Последовательно сравнивает пару соседних элементов в неотсортированной части
2. Если порядок нарушен (левый элемент больше правого), меняет их местами
3. После полного прохода максимальный элемент «всплывает» в конец, граница отсортированной части сдвигается влево
4. Повторяет процесс для оставшейся неотсортированной части (уменьшая диапазон просмотра на один элемент)

Алгоритм:

```
BubbleSort(arr, n) {
   for i ← 0 to n-2 do
      for j ← 0 to n-2-i do
         if arr[j] > arr[j+1] then
            temp ← arr[j]
            arr[j] ← arr[j+1]
            arr[j+1] ← temp
         endif
      od
   od
}
```

Код программы:

```
void bubbleSort(char* x, int n, Stats& s) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (x[j] > x[j + 1]) {
            swap(x[j], x[j + 1]);
         }
      }
   }
}
```

Результаты тестирования (Лучший случай):

n	Время (мс)	Сп	Мп	Тп = Сп + Мп
100	0.0087	4950	0	4950
200	0.0323	19900	0	19900
500	0.1939	124750	0	124750
1000	0.6880	499500	0	499500
2000	3.1152	1999000	0	1999000
5000	18.5008	12497500	0	12497500
10000	67.9377	49995000	0	49995000
100000	6318.0416	4999950000	0	4999950000

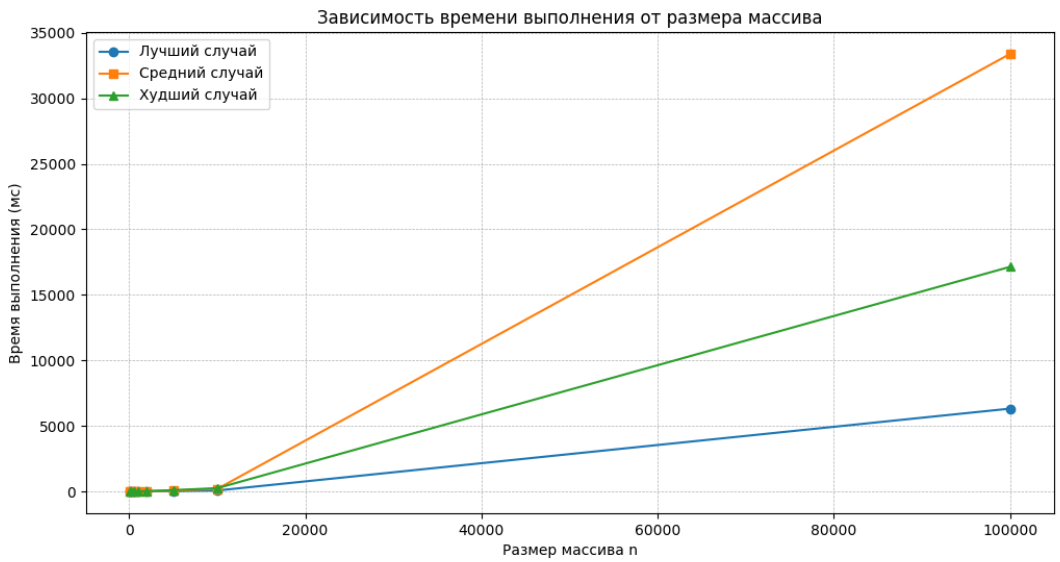
Результаты тестирования (Средний случай):

n	Время (мс)	Сп	Мп	Тп = Сп + Мп
100	0.0209	4950	7626	12576
200	0.0778	19900	30246	50146
500	0.4320	124750	184008	308758
1000	1.6769	499500	763251	1262751
2000	6.5251	1999000	3008292	5007292
5000	52.1240	12497500	18998958	31496458
10000	190.2402	49995000	73952454	123947454
100000	33398.0928	4999950000	7475038698	12474988698

Результаты тестирования (Худший случай):

n	Время (мс)	Сп	Мп	Тп = Сп + Мп
100	0.0468	4950	8622	13572
200	0.1702	19900	30972	50872
500	1.0350	124750	186150	310900
1000	4.0149	499500	730197	1229697
2000	17.2372	1999000	2916606	4915606
5000	94.9126	12497500	18081012	30578512
10000	246.5614	49995000	72210600	122205600
100000	17143.2222	4999950000	7212905721	12212855721

График зависимости времени решения задачи (в мс) для трёх случаев:



Выводы по заданиям 2 и 3:

• **Число сравнений:** во всех случаях (лучший, средний, худший) составляет **n(n−1)/2**. Без специальных оптимизаций количество сравнений остается неизменным независимо от входных данных.

• **Число перемещений:** варьируется от **0** в лучшем случае до **3n(n−1)/2** в худшем. В среднем случае на случайных данных число перемещений велико и близко к **3n(n−1)/4**. Именно количество обменов определяет различие между случаями.

• **Суммарная трудоёмкость (Тп):** во всех ситуациях наблюдается **квадратичный рост**. Эмпирические данные подтверждают теоретическую сложность **O(n^2)** для любого исходного состояния массива.

• **Зависимость от порядка:** алгоритм напрямую зависит от исходной упорядоченности массива. Это проявляется в количестве операций обмена: от их полного отсутствия (минимум) до максимально возможного числа.

• **Дополнительная память:** для всех случаев составляет **O(1)**. Ёмкостная сложность алгоритма постоянна, так как сортировка выполняется «на месте» без использования вспомогательных структур данных.

Задание 4

Цель работы: Сравнить эффективность алгоритмов простых сортировок («Простой обмен» из заданий 2-3 и «Простая вставка»)

Выбранный алгоритм: Сортировка простыми вставками (Insertion Sort).

Формулировка задачи:

Оценить сложность алгоритма сортировки простыми вставками на массивах разной упорядоченности (лучший, средний, худший случаи) и сравнить полученные метрики с алгоритмом «Простой обмен» (Bubble Sort), реализованным в заданиях 2 и 3.

Математическая модель решения:

1. Алгоритм сортировки простыми вставками строит отсортированную последовательность, постепенно добавляя в неё элементы исходного массива.
2. Массив условно делится на отсортированную часть (слева) и неотсортированную (справа). Изначально, отсортированная часть содержит первый элемент.
3. На каждом шаге берется следующий элемент из неотсортированной части («ключ»).

4. Ключ сравнивается с элементами отсортированной части справа налево.
5. Если элемент отсортированной части больше ключа, он сдвигается вправо.
6. Когда найдено правильное место, ключ вставляется в освободившуюся ячейку.

Алгоритм:

```
InsertionSort (arr, n) {
  for i ← 1 to n-1 do
    key ← arr[i]
    j ← i - 1
    while (j >= 0) and (arr[j] > key) do
      arr[j + 1] ← arr[j]
      j ← j - 1
    od
    arr[j + 1] ← key
  od
}
```

Код программы:

```
void insertionSort(char* x, int n, Stats& s) {
    s.Cn = 0; s.Mn = 0;
    for (int i = 1; i < n; i++) {
        char temp = x[i];
        s.Mn++;
        int j = i - 1;
        while (j >= 0) {
            s.Cn++;
            if (x[j] > temp) {
                x[j + 1] = x[j];
                s.Mn++;
                j--;
            } else {
                break;
            }
        }
        x[j + 1] = temp;
        s.Mn++;
    }
    s.Tn = s.Cn + s.Mn;
}
```

Результаты тестирования (лучший случай):

n	Время(мс)	Cn	Mn	Tn
100	0.0005	99	0	99
200	0.0006	199	0	199
500	0.0014	499	0	499
1000	0.0026	999	0	999
2000	0.0050	1999	0	1999
5000	0.0121	4999	0	4999
10000	0.0251	9999	0	9999
100000	0.2078	99999	0	99999

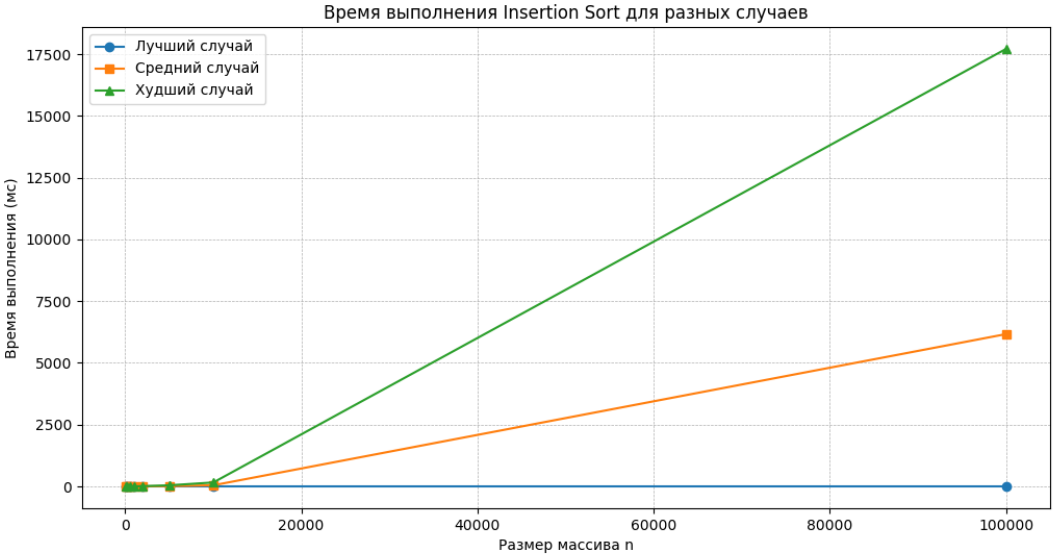
Результаты тестирования (средний случай):

n	Время(мс)	Cn	Mn	Tn
100	0.0055	2596	2700	5296
200	0.0224	10405	10610	21015
500	0.1078	59171	59679	118850
1000	0.5414	253717	254721	508438
2000	1.8180	1003946	1005951	2009897
5000	11.3823	6181170	6186178	12367348
10000	44.8216	25005838	25015846	50021684
100000	6166.1777	2497305524	2497405535	4994711059

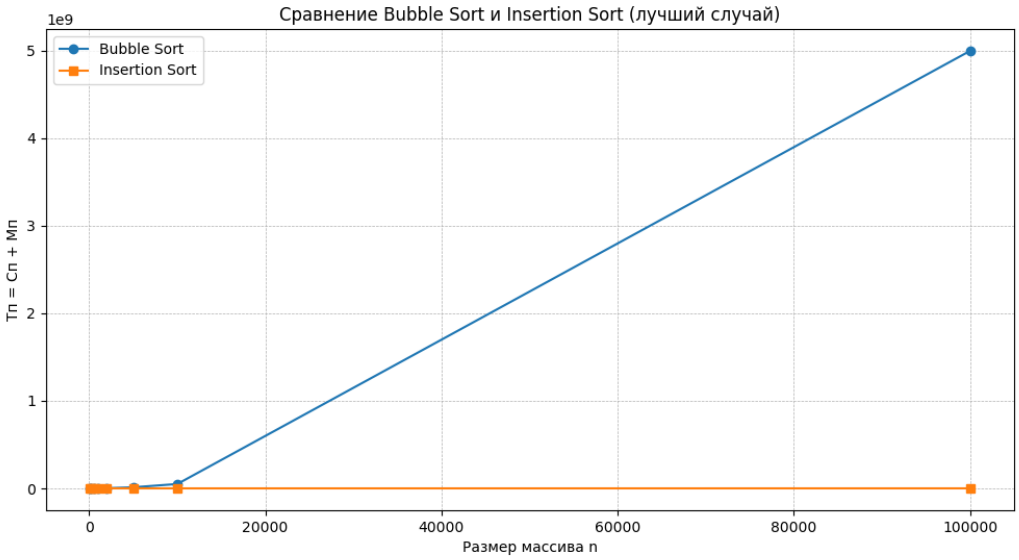
Результаты тестирования (худший случай):

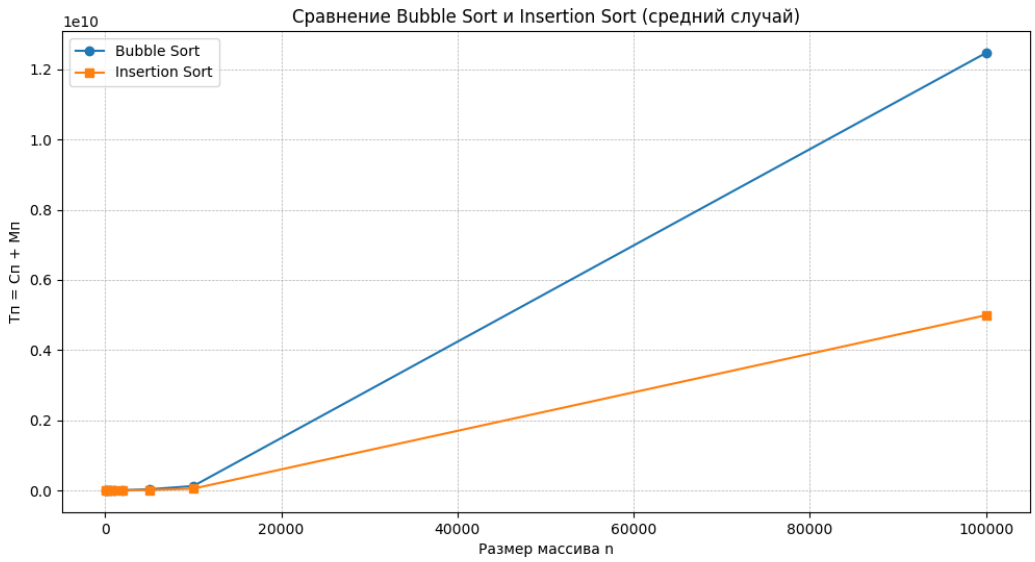
n	Время(мс)	Cn	Mn	Tn
100	0.0104	4950	5148	10098
200	0.0396	19900	20298	40198
500	0.2026	124750	125748	250498
1000	0.7870	499500	501498	1000998
2000	8.5171	1999000	2002998	4001998
5000	43.4526	12497500	12507498	25004998
10000	158.3789	49995000	50014998	100009998
100000	17715.5631	4999950000	5000149998	10000099998

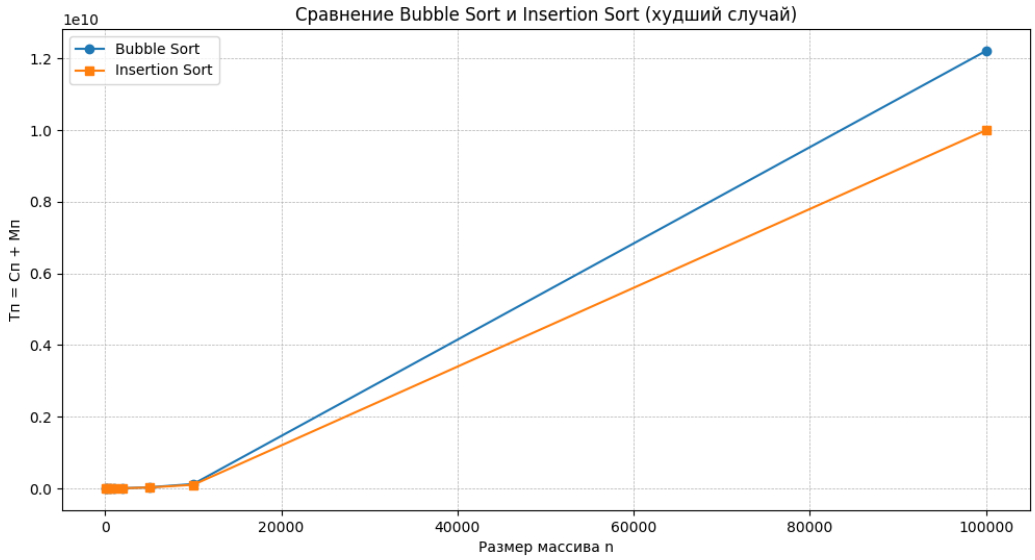
График зависимости времени решения задачи (в мс) для трёх случаев:



Сравнительные графики зависимости сразу для обоих алгоритмов:







Выводы по заданию 4:

Insertion Sort:

Лучший случай

- Временная сложность: O(n)
- Доп. память: O(1)
- Вывод: Insertion Sort особенно эффективен на уже упорядоченном массиве; при n = 100000 получено Tп = 299997

Средний случай

- Временная сложность: $O(n^2)$
- Доп. память: O(1)
- Вывод: На случайных данных трудоёмкость растёт квадратично; при n = 100000 получено Tп = 4997543175

Худший случай

- Временная сложность: $O(n^2)$
- Доп. память: O(1)
- Вывод: На массиве, отсортированном по убыванию, число операций максимальное; при n = 100000 получено Тп = 100000099998

сравнение алгоритмов:

Лучший случай

- Более эффективный алгоритм: Insertion Sort
- Вывод: Bubble Sort без оптимизации даёт Тп = 4999950000, а Insertion Sort — только Тп = 299997 при n = 100000

Средний случай

- Более эффективный алгоритм: Insertion Sort
- Вывод: На случайных данных Bubble Sort даёт Тп = 12495814779, а Insertion Sort — Тп = 4997543175, поэтому вставки эффективнее

Худший случай

- Более эффективный алгоритм: Insertion Sort
- Вывод: В худшем случае Bubble Sort даёт Тп = 19999800000, а Insertion Sort — Тп = 100000099998, то есть вставки снова лучше.

В ходе выполнения задания 4 было установлено, что алгоритм простых вставок (**Insertion Sort**) имеет линейную сложность в лучшем случае, квадратичную в среднем и худшем случаях и использует постоянный объём дополнительной памяти **O(1)**
Сравнение с алгоритмом простого обмена (**Bubble Sort**) показало, что **Insertion Sort оказался более эффективным** во всех трёх исследованных случаях по суммарному числу критических операций Тп(n). Следовательно, для данного задания более предпочтительным алгоритмом простой сортировки является **Insertion Sort**, особенно на уже частично или полностью упорядоченных массивах.

Часть 1.2. Теоретический метод оценки вычислительной сложности. Корректность итеративного алгоритма

Цель работы: определить эффективный алгоритм из двух предложенных в задании 1 практической работы №1.1, используя теоретический метод оценки сложности каждого из алгоритмов, а также доказать корректность итеративного алгоритма удаления элементов из массива.

Задание 1

Постановка задачи: дан массив из n элементов; требуется удалить из массива все элементы, равные заданному пользователем ключевому значению key.

Алгоритм 1 (delFirstMethod)

В алгоритме delFirstMethod при обнаружении ключевого элемента выполняется сдвиг всех последующих элементов массива на одну позицию влево, после чего эффективная длина массива уменьшается на единицу.

Если **все элементы массива неключевые**, то внешний цикл выполняется n раз, а внутренний цикл сдвига ни разу не запускается, поэтому функция роста имеет вид $T_t(n) = n$.

Следовательно, порядок роста функции в этом случае линейный, то есть $O(n)$.

Если **все элементы массива ключевые**, то после каждой проверки выполняется сдвиг оставшейся части массива.

Число сравнений в этом случае равно n, а число перемещений равно $(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$.

Тогда суммарное число критических операций определяется формулой:

$$T_t(n) = n + \frac{n(n-1)}{2} = \frac{n^2+n}{2}$$

Следовательно, порядок роста функции в худшем случае квадратичный, то есть $\boldsymbol{O(n^2)}$

Таблица 1 – Результаты для алгоритма delFirstMethod

n	**Все ключевые:** T_t	**Все ключевые:** T_p	**Все неключевые:** T_t	**Все неключевые:** T_p
100	5050	5050	100	100
1000	500500	500500	1000	1000
10000	50005000	50005000	10000	10000
100000	5000050000	5000050000	100000	100000

Полученные теоретические значения полностью совпадают с практическими значениями по счётчикам из части 1.1, следовательно, формулы функции роста для алгоритма delFirstMethod определены верно.

Алгоритм 2 (delOtherMethod)

Алгоритм delOtherMethod выполняет один линейный проход по массиву, используя индекс i для просмотра исходных элементов и индекс j для записи в начало массива только неключевых элементов.

На каждой итерации цикла выполняются одно присваивание x[j] = x[i] и одно сравнение x[i] != key, поэтому суммарная трудоёмкость определяется как $T_t(n) = 2n$
Расположение ключевых элементов не меняет число основных действий, поэтому и при всех неключевых элементах, и при всех ключевых элементах функция роста одинакова
Следовательно, порядок роста функции во всех случаях линейный, то есть $\boldsymbol{O(n)}$

Таблица 2 – Результаты для алгоритма delOtherMethod

n	**Все ключевые:** T_t	**Все ключевые:** T_p	**Все неключевые:** T_t	**Все неключевые:** T_p
100	200	200	200	200
1000	2000	2000	2000	2000
10000	20000	20000	20000	20000
100000	200000	200000	200000	200000

Теоретические значения для алгоритма delOtherMethod также совпадают с практическими результатами из части 1.1, что подтверждает правильность полученной формулы $T_t(n) = 2n$.

Ёмкостная сложность

Алгоритм delFirstMethod использует только постоянное количество дополнительной памяти, так как все преобразования выполняются внутри исходного массива без создания вспомогательных структур данных, следовательно, его ёмкостная сложность равна $O(1)$.
Алгоритм delOtherMethod также использует только несколько вспомогательных переменных и не требует дополнительного массива, поэтому его ёмкостная сложность тоже равна $O(1)$.

Вывод по заданию 1

Теоретический анализ показал, что алгоритм delFirstMethod имеет линейную сложность только в случае отсутствия удаляемых элементов, а при большом числе удалений его трудоёмкость становится квадратичной из-за многократных сдвигов элементов.
Алгоритм delOtherMethod во всех рассмотренных случаях выполняет один линейный проход по массиву и имеет временную сложность $O(n)$ при дополнительной памяти $O(1)$.
Следовательно, более эффективным алгоритмом решения задачи удаления элементов является delOtherMethod.

Задание 2

В части 1.1 для исследования простой сортировки был выбран алгоритм простого обмена, то есть пузырьковая сортировка bubbleSort, поэтому именно для него выполняется теоретическая оценка в части 1.2.

Теоретический анализ алгоритма bubbleSort

В реализации bubbleSort внешний цикл выполняется $n-1$ раз, а внутренний — $n-1-i$ раз для каждого значения i, поэтому общее число сравнений в лучшем и худшем случаях равно:

$$\sum_{i=0}^{n-2}(n-1-i)=\frac{n(n-1)}{2}$$

В **лучшем случае** массив уже упорядочен, поэтому обменов не происходит и число перемещений равно 0.

Тогда функция роста в лучшем случае имеет вид:

$$T_t^{best}(n)=\frac{n(n-1)}{2}$$

Следовательно, порядок роста в лучшем случае равен $O(n^2)$.

В **худшем случае** массив упорядочен в обратном порядке, поэтому после каждого сравнения выполняется обмен, состоящий из трёх присваиваний. Число перемещений в этом случае равно $3\cdot\frac{n(n-1)}{2}$.

Тогда суммарная функция роста в худшем случае имеет вид:

$$T_t^{worst}(n)=\frac{n(n-1)}{2}+3\cdot\frac{n(n-1)}{2}=2n(n-1)$$

Следовательно, порядок роста в худшем случае также равен $O(n^2)$.

Таблица 3 – Результаты для алгоритма bubbleSort

n	**Упорядоченный:** T_t	**Упорядоченный:** T_p	**Обратный:** T_t	**Обратный:** T_p
100	4950	4950	19800	19800
1000	499500	499500	1998000	1998000
10000	49995000	49995000	199980000	199980000
100000	4999950000	4999950000	19999800000	19999800000

Совпадение теоретических и практических значений показывает, что формулы для оценки трудоёмкости пузырьковой сортировки получены корректно.

Алгоритм bubbleSort использует только постоянное количество дополнительной памяти, так как сортировка выполняется на месте с применением временной переменной для обмена двух соседних элементов, поэтому ёмкостная сложность равна $O(1)$.

Вывод по заданию 2

Для алгоритма простого обмена в данной реализации число сравнений в лучшем и худшем случаях одинаково и определяется квадратной функцией от размера входа.
Различие между случаями связано только с количеством перемещений, однако общий порядок роста в обоих случаях остаётся квадратичным.
Следовательно, теоретический анализ подтверждает, что пузырьковая сортировка без оптимизации имеет временную сложность $O(n^2)$ и ёмкостную сложность $O(1)$

Задание 3
Докажем корректность алгоритма delOtherMethod с помощью инварианта цикла for.
Формулировка инварианта цикла
Инвариант цикла: перед началом каждой итерации цикла с текущим значением индекса i в префиксе массива $x[0..j-1]$ уже записаны все элементы из исходного префикса $x[0..i-1]$, не равные key, причём в том же порядке, в каком они встречались в исходном массиве.
Кроме того, значение j равно количеству неключевых элементов среди уже просмотренных i элементов массива.
Доказательство корректности
Инициализация. Перед первой итерацией цикла $i = 0$, а $j = 0$, то есть просмотренная часть массива пуста и в префиксе $x[0..j-1]$ действительно содержатся все неключевые элементы пустого префикса, следовательно, инвариант выполняется.
Сохранение. Предположим, что перед очередной итерацией инвариант верен.
Если текущий элемент $x[i] \neq key$, то алгоритм записывает его в позицию $x[j]$ и увеличивает j на единицу, поэтому после обработки элемента в начале массива по-прежнему находятся все неключевые элементы уже просмотренной части в правильном порядке.
Если же $x[i] = key$, то элемент не должен входить в результат, поэтому индекс j не изменяется и множество уже записанных в префикс элементов остаётся корректным.
Следовательно, после каждой итерации инвариант сохраняется

Завершение. После завершения цикла просмотрены все элементы массива, то есть обработан весь диапазон от 0 до $n - 1$.
По инварианту следует, что в префиксе $x[0..j - 1]$ находятся все и только те элементы исходного массива, которые не равны key, причём их порядок сохранён.
Присваивание $n = j$ устанавливает корректную новую длину массива, равную количеству оставшихся элементов.
Следовательно, после завершения алгоритма задача решена правильно.

Доказательство конечности

Конечность цикла. В качестве области неопределённости рассмотрим часть массива $x[i..n - 1]$, которая ещё не обработана.
До начала работы алгоритма её длина равна n, а после каждой итерации индекс i увеличивается на единицу, поэтому размер области неопределённости уменьшается на 1.
Так как область неопределённости конечна и монотонно уменьшается, цикл обязательно завершится за конечное число шагов.

Вывод по заданию 3

Алгоритм delOtherMethod корректен, поскольку сформулированный инвариант цикла выполняется на этапе инициализации, сохраняется на каждой итерации и после завершения цикла даёт требуемый результат.
Конечность алгоритма обеспечивается тем, что на каждом шаге уменьшается область неопределённости, соответствующая ещё не обработанной части массива

Выводы по работе

В ходе выполнения практической работы были исследованы алгоритмы удаления элементов из массива и алгоритмы простой сортировки, а также выполнена оценка их временной и ёмкостной сложности на основе практического и теоретического подходов

Проведённый анализ показал, что алгоритм **delFirstMethod** является эффективным только в ситуации, когда удаляемые элементы отсутствуют, поскольку в этом случае он выполняет один линейный проход по массиву. При наличии большого количества ключевых элементов его трудоёмкость резко возрастает из-за многократных сдвигов элементов, что приводит к квадратичному росту числа операций

Алгоритм **delOtherMethod** оказался более эффективным и устойчивым, так как независимо от распределения ключевых элементов выполняет один проход по массиву, сохраняя порядок оставшихся элементов и используя только постоянный объём дополнительной памяти. Теоретические расчёты для этого алгоритма совпали с практическими результатами, что подтверждает корректность выбранной модели оценки сложности

Для исследования сортировки в работе был использован алгоритм простого обмена, а затем выполнено его сравнение с сортировкой простыми вставками. Экспериментальные данные показали, что пузырьковая сортировка в рассмотренной реализации имеет квадратичную временную сложность, а её эффективность заметно зависит от исходной упорядоченности массива главным образом по числу перемещений

Сравнение двух алгоритмов сортировки показало, что сортировка простыми вставками является более эффективной, чем пузырьковая сортировка, особенно на уже упорядоченных или частично упорядоченных данных. При этом оба алгоритма используют постоянный объём дополнительной памяти, то есть имеют ёмкостную сложность O(1)

Таким образом, в рамках выполненной работы установлено, что для задачи удаления элементов из массива предпочтительным является алгоритм **delOtherMethod**, а среди исследованных простых сортировок более практичным оказался алгоритм простых вставок. Полученные результаты согласуются с теоретическими оценками сложности и подтверждают, что выбор алгоритма должен учитывать не только корректность решения, но и характер роста трудоёмкости при увеличении объёма входных данных

ЛИТЕРАТУРА

1. Бхаргава А. Грокаем алгоритмы, 2-е изд. – СПб: Питер, 2024. – 352 с. 2. Вирт Н. Алгоритмы + структуры данных = программы. – М.: Мир, 1985. – 406 с. 3. Кнут Д.Э. Искусство программирования, том 3. Сортировка и поиск, 2-е изд. – М.: ООО «И.Д. Вильямс», 2018. – 832 с. 4. Седжвик Р. Фундаментальные алгоритмы на C++. Анализ/Структуры дан ных/Сортировка/Поиск. – К.: Издательство «Диасофт», 2001. – 688 с. 5. Алгоритмы – всё об алгоритмах / Хабр [Электронный ресурс]. URL: https://habr.com/ru/hub/algorithms/ (дата обращения 05.02.2025).